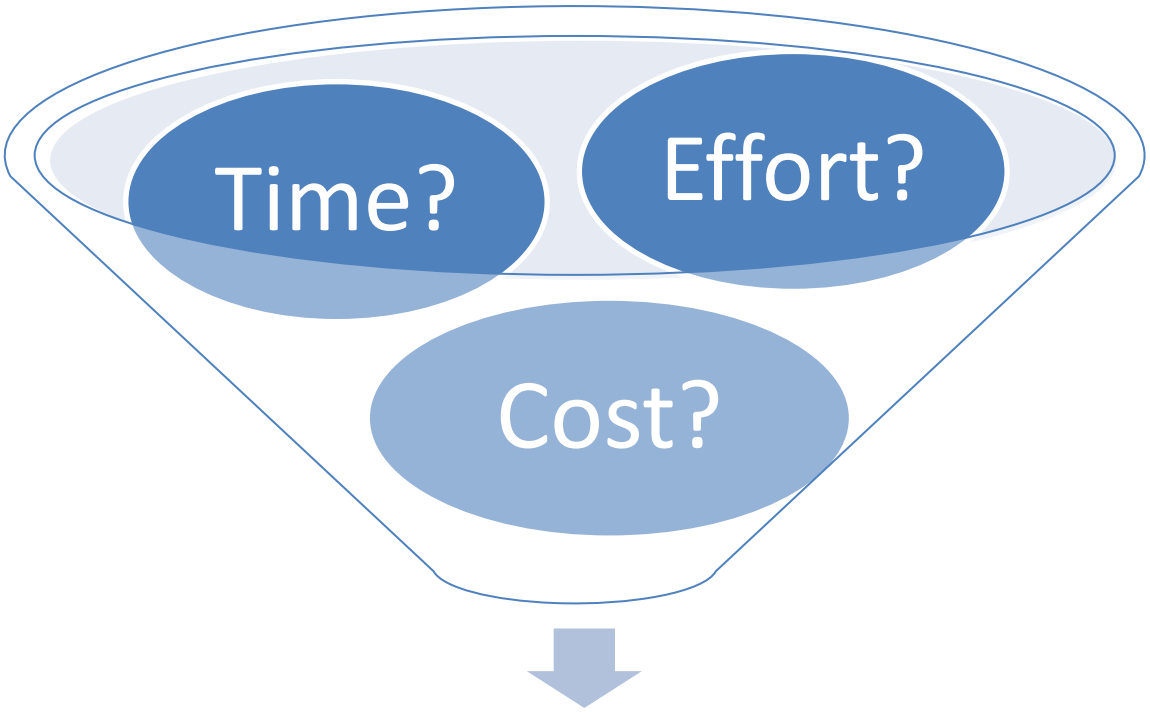
Project Management
Effort and size Estimation
Part -1

Fundamental estimation questions



Fundamental estimation questions

- How much effort is required to complete an activity?
- How much calendar time is needed to complete an activity?
- What is the total cost of an activity?
- Project estimation and scheduling are interleaved management activities.

Software Project Estimation Effort Estimation

It Is the process of predicting Effort, productivity, time and cost required to develop software projects.

It uses Software metrics to estimate size of projects

Software Metrics refers to measurements relating to computer software

Estimating Model

Software Estimation process…

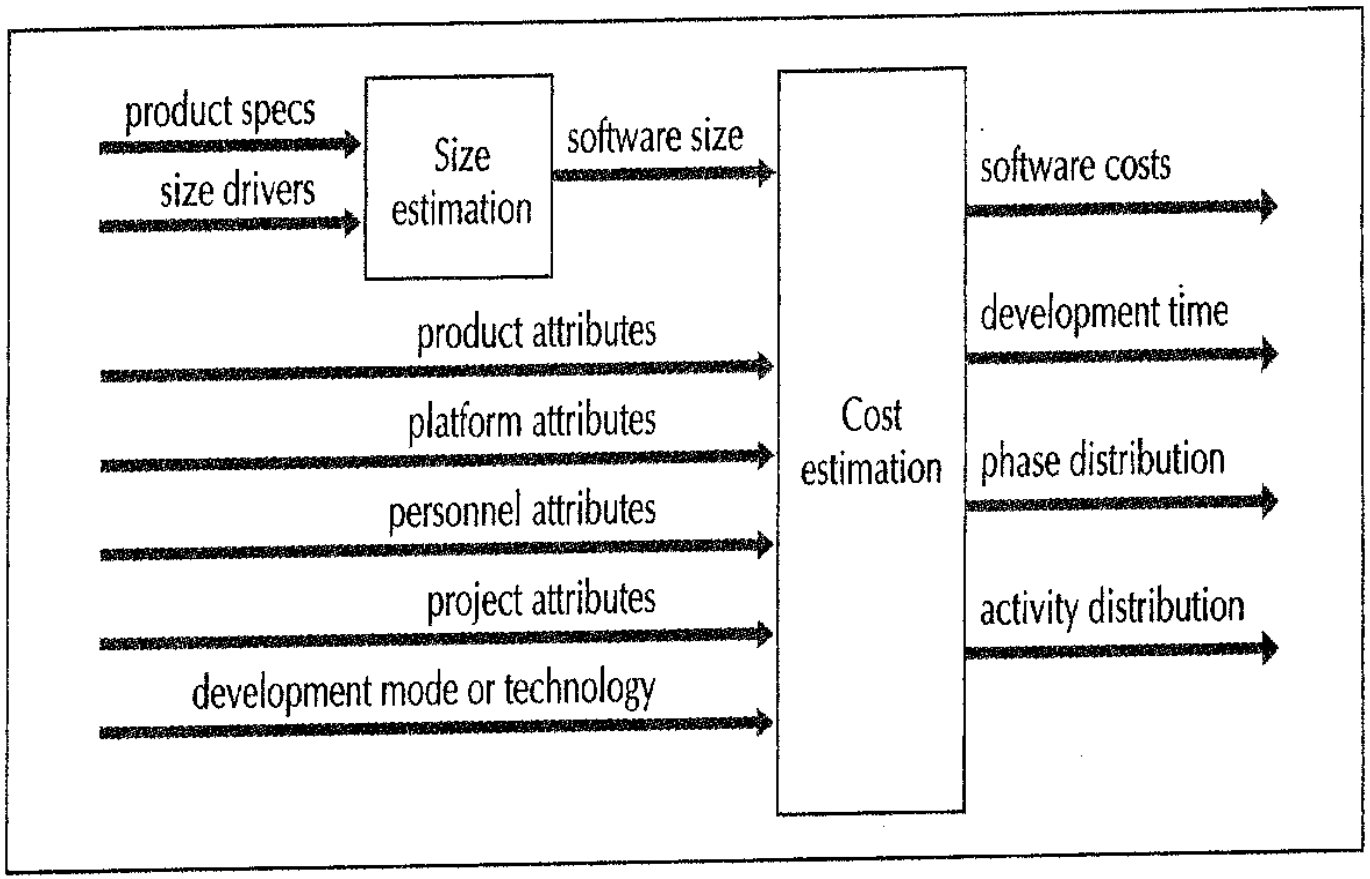


FIGURE 3.7

The general cost estimation process

Software Effort Estimation

- It Is the process of predicting Effort, productivity, time and cost required to develop software projects.
- Effort can be calculated as Staff or Person per day/Week/Month/Year.
- For small project it is common to use Staff-Weeks while for large project it is common to use Staff-months

The four basic steps in software project estimation are:

1) Estimate the size of the development product.

   This generally ends up in either Lines of Code (LOC) or Function Points (FP), but there are other possible units of measure. A discussion of the pros & cons of each is discussed in some of the material referenced at the end of this report.

2) Estimate the effort in person-months or person-hours.

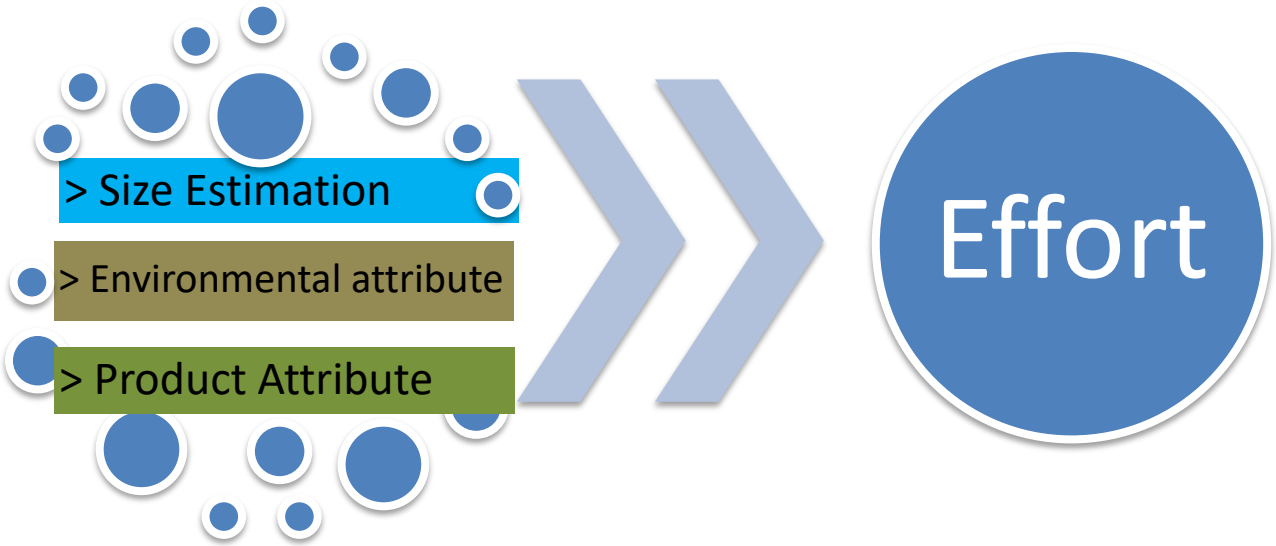
3) Estimate the schedule in calendar months.

4) Estimate the project cost in dollars (or local currency)

Effort

- **Effort can be calculated as Staff or Person per day/Week/Month/Year**
- **For small project it is common to use Staff-Weeks while for large project it is common to use Staff-months**
- **Example 30 staff-month means that you need 30 staff to finish it in one month**
- **For two months you need only 15 staff per month (30/2) you have more time so you need less people**
- **To calculate staff-month in weeks you need to multiply by 4 30*4 = 120 staff-week; to calculated in years /12 (30/12= 2.5 staff-year)**
- **To calculate effort phase percentage (e.g. 10% of total effort) 10% of 30 = 3 staff-month. If for example the phase last for 3 months then you need 1 staff for each month 3/3)**

Software Estimation process…



A taxonomy of estimating methods

- Bottom-up - activity based, analytical
- Parametric or algorithmic models e.g. function points
- Expert opinion - just guessing?
- Analogy - case-based, comparative
- Parkinson and ‘price to win’ ?? (Search it)

Bottom-up versus top-down

- **Bottom-up**
  - use when no past project data
  - identify all tasks that have to be done – so quite time-consuming
  - use when you have no data about similar past projects
- **Top-down**
  - produce overall estimate based on project cost drivers
  - based on past project data
  - divide overall estimate between jobs to be done

Bottom-up estimating

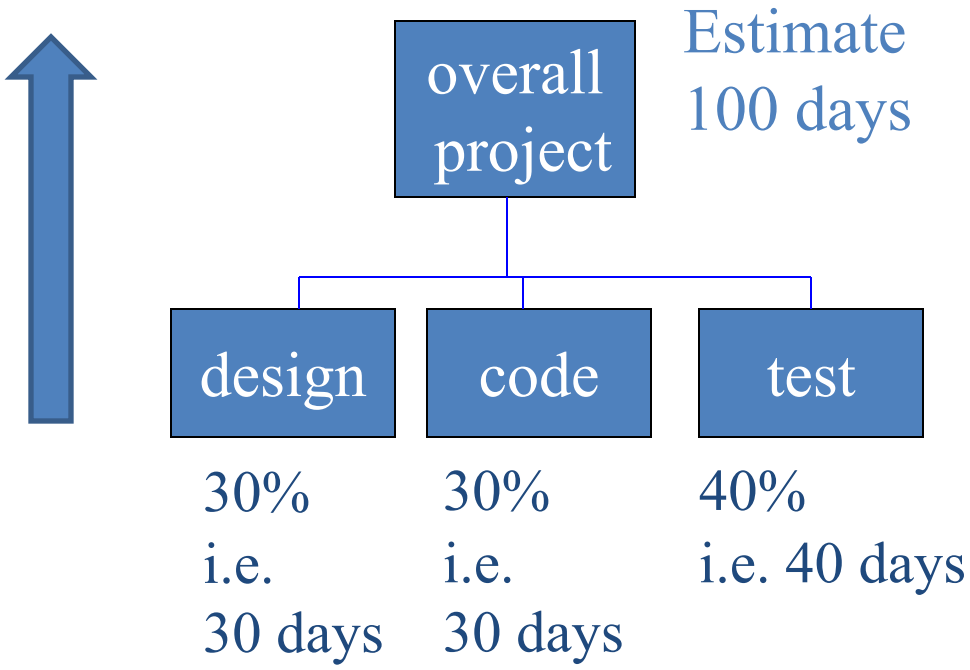
1. Break project into smaller and smaller components

[2. Stop when you get to what one person can do in one/two weeks]

3. Estimate costs for the lowest level activities

4. At each higher level calculate estimate by adding estimates for lower levels

Bottom-up estimating

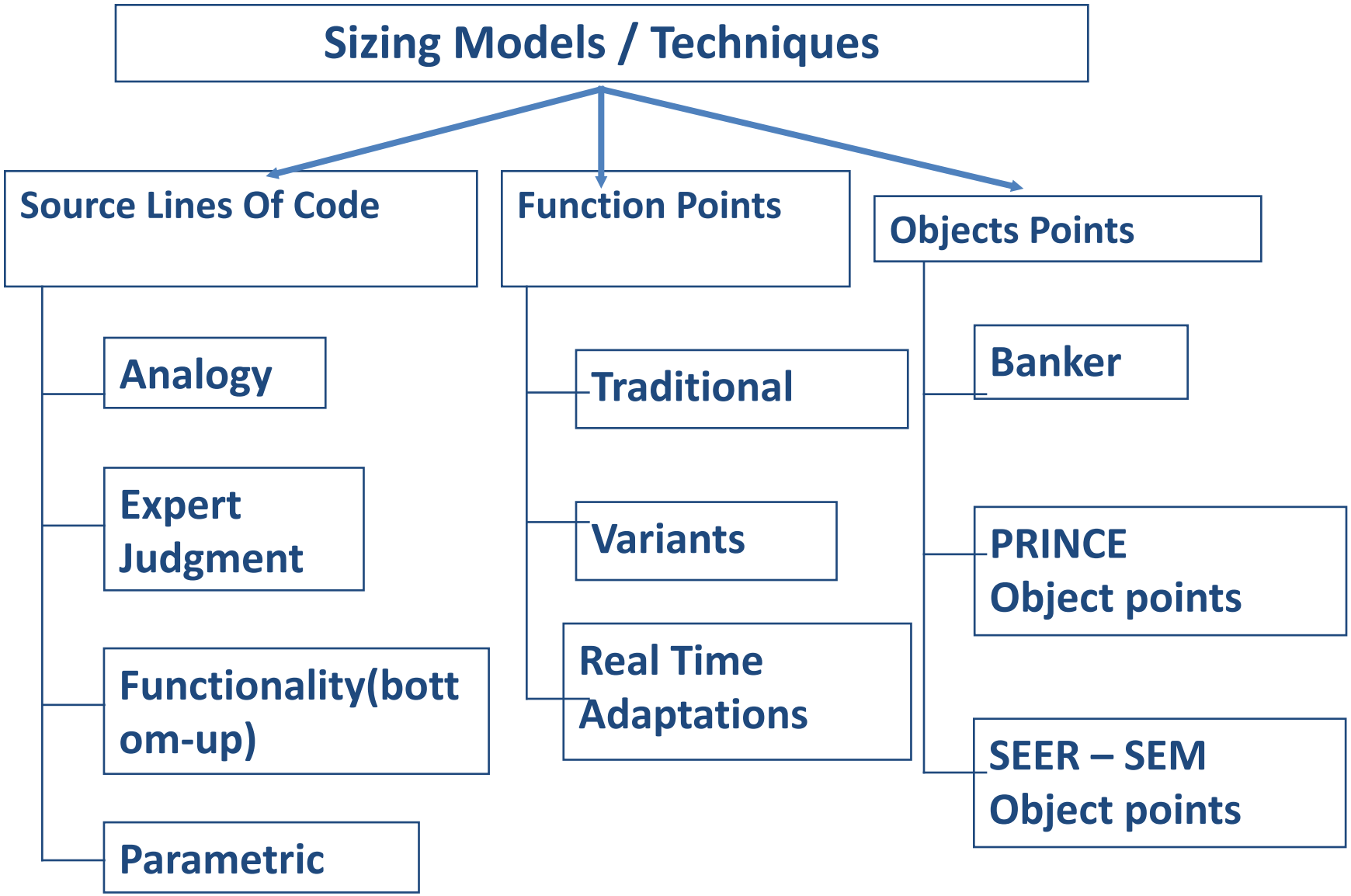


- Produce overall estimate using effort driver(s)
- distribute proportions of overall estimate to components

Sizing the project

- Size is a primary cost factor in most models.
  - Development time is a function of program size
  - the time it takes to develop a program depends upon the size (and the complexity) of the program
  - Product size measures should correlate to the effort/ hours you need to develop them.
  - There are several common types of code size

    as shown in the next diagram.


Sizing Models / Techniques
Source Lines Of Code
Function Points
Objects Points
Analogy
Expert Judgment
Functionality(bottom-up)
Parametric
Traditional
Variants
Real Time Adaptations
Banker
PRINCE Object points
SEER – SEM Object points

Source Line of Code (LOC)

- LOC = NCLOC + CLOC
  - NCLOC : non- commented Line of code.
  - CLOC :commented Line of code.

- KLOC is used to denote thousands of Line of code.
- Blank lines - not usually counted

Example

```
for (i = 0; i < 100; i++) printf("hello"); /* How many lines of code is this? */
```

In this example we have:

- 1 Physical Line of Code (LOC)
- 2 Logical Lines of Code (LLOC) (for statement and printf statement)
- 1 comment line

Example 2

```
/* Now how many lines of code is this? */
for (i = 0; i < 100; i++)
 {
printf("hello");
 }
```

In this example we have:

- 5 Physical Lines of Code (LOC): is placing braces work to be estimated?
- 2 Logical Line of Code (LLOC): what about all the work writing non-statement lines?
- 1 comment line: tools must account for all code and comments regardless of comment placement.

SLOC

- Physical LOC: counts text lines
- Logical LOC: counts language elements(if, then, else, begin, end, …) :
- First example
  - Logical LOC : 3 LOC
  - Physical LOC: 3 LOC
- Second example:
  - Logical LOC: 7 LOC
  - Physical lines :9 LOC

- Example 1

```
1. If A>B
2.     then A - B
3. else A + B;
```

- Example 2

```
1. If A>B
2. then
3. begin
4.     A - B
5. end
6. Else
7. begin
8.     A + B
9. end;
```

SLOC

How many physical source lines are there in this C language program?

```
#define  LOWER 0  /* lower limit of table */
#define  UPPER 300      /* upper limit */
#define  STEP 20  /* step size */

main()       /* print a Fahrenheit->Celsius conversion
table */
{
  int fahr;
  for(fahr=LOWER; fahr<=UPPER; fahr=fahr+STEP)
      printf("%4d %6.1f\n", fahr, (5.0/9.0)*(fahr-32));
}
```

SLOC.. Expert judgment model

$$S = (a + 4m + b) / 6$$

- *"**S**" is SLOC,*
- *"**a**' is the smallest possible size (optimistic)*
- *"**m**" is the most likely size*
- *"**b**" is the largest possible size. (Pessimistic)*
- *"**a**","**m**","**b**" is determined by experts.*

Estimation of LOC

Consider the following software project

S/W Component		a	m	b	LOC
User interface and control facilities	UICF	1,500	2,300	3,100	2,300
Two-dimensional geometric analysis	2DGA	3,800	5,200	7,200	5,300
Three-dimensional geometric analysis	3DGA	4,600	6,900	8,600	6,800
Database management	DBM	1,600	3,500	4,500	3,350
Computer graphics display features	CGDF	3,700	5,000	6,000	4,950
Peripheral control	PC	1,400	2,200	2,400	2,100
Design analysis modules	DAM	7,200	8,300	10,000	8,400
Estimated lines of code		23,800	33,400	41,800	33,200

What are the drawbacks of SLOC?

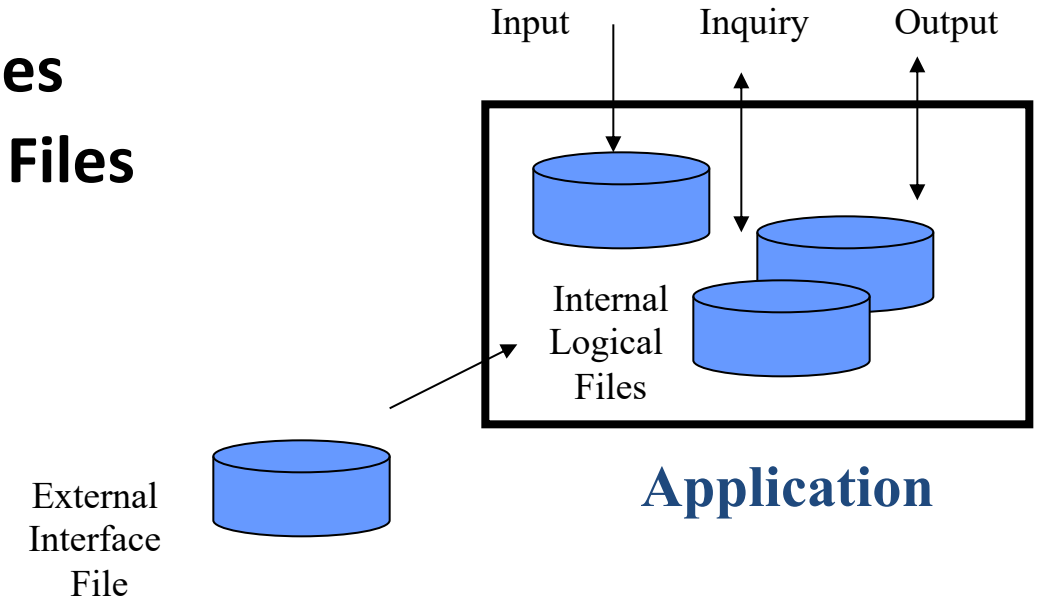
Function Points

- A measure of the size of computer applications
- The size is measured from a functional, or user, point of view.
- It is independent of the computer language, development methodology, technology or capability of the project team used to develop the application.
- Can be subjective
- Can be estimated EARLY in the software development life cycle.

The Function Point Methodology

- **Inputs**
- **Outputs**
- **Inquiries**
- **Internal Logical Files**
- **External Interface Files**

Five key components are identified based on logical user view



Function Points - continued

Five function types

1. **Logical interface file (LIF)** types – equates roughly to a datastore in systems analysis terms. Created and accessed by the target system.

2. **External interface file types (EIF)** – where data is retrieved from a datastore which is actually maintained by a different application.

Function Points - continued

3. **External input (EI) types** – input transactions which update internal computer files.

4. **External output (EO) types** – transactions which extract and display data from internal computer files. Generally involves creating reports.

5. **External inquiry (EQ) types** – user initiated transactions which provide information but do not update computer files. Normally the user inputs some data that guides the system to the information the user needs.

FP complexity multipliers

External user types	Low complexity	Medium complexity	High complexity
EI	_____x3	_____x4	_____x6
EO	_____x4	_____x5	_____x7
EQ	_____x3	_____x4	_____x6
LIF	_____x7	_____x10	_____x15
EIF	_____x5	_____x7	_____x10

- Unadjusted FP Count (UFPC)= Sum (sum of product for each type).

Example(1)

Payroll application has:

1. **Transaction to input, amend and delete employee details – an EI that is rated of medium complexity**
2. **A transaction that calculates pay details from timesheet data that is input – an EI of high complexity**
3. **A transaction of medium complexity that prints out pay-to-date details for each employee – EO**
4. **A file of payroll details for each employee – assessed as of medium complexity LIF**
5. **A personnel file maintained by another system is accessed for name and address details – a simple EIF**

What would be the FP counts for these?

FP counts

1.	Medium EI	4 FPs
2.	High complexity EI	6 FPs
3.	Medium complexity EO	5 FPs
4.	Medium complexity LIF	10 FPs
5.	Simple EIF	5 FPs
Total		**30 FPs**

If previous projects delivered 5 FPs a day, implementing the above should take 30/5 = 6 days

Example (2)

- **LIF** — **4 easy 5 average 3 Hard**
- **EI** — **7 easy 6 average 2 Hard**
- **EO** — **6 easy 2 average 3 Hard**
- **EQ** — **3 easy 4 average 2 Hard**

External user types	Easy	Average	Hard	UFP
EI	**7**x3	**6**x4	**2**x6	**57**
EO	**6**x4	**2**x5	**3**x7	**55**
EQ	**3**x3	**4**x4	**2**x6	**37**
LIF	**4**x7	**5**x10	**3**x15	**123**
EIF	**0**x5	**0**x7	**0**x10	**0**
Unadjusted Function Point Counts (**UFPC**)				**272**

Calculate Degree of Influence (DI)

1. Does the system require reliable backup and recovery? 3
2. Are data communications required? 4
3. Are there distributed processing functions? 1
4. Is performance critical? 3
5. Will the system run in an existing, heavily utilized operational environment? 2
6. Does the system require on-line data entry?
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations? 4
8. Are the master files updated on-line? 3
9. Are the inputs, outputs, files, or inquiries complex? 3
10. Is the internal processing complex? 2
11. Is the code designed to be reusable? 1
12. Are conversion and installation included in the design? 3
13. Is the system designed for multiple installations in different organizations? 5
14. Is the application designed to facilitate change and ease of use by the user? 1

1

The FP Calculation:

- Inputs include:
  - Count Total
  - DI = ΣF_i (i.e., sum of the Adjustment factors F_1.. F_{14})
- Calculate Function points using the following formula:
  FP = UFPC X [0.65 + 0.01 X Σ *Fi*]

TCF: Technical complexity factor

- In this example:
  FP = 345 X [0.65 + 0.01 X (3+4+1+3+2+4+3+3+2+1+3+5+1+1)
  FP = 345 X [0.65 + 0.01 X (36)
  FP = 345 X [0.65 + 0.36]
  FP = 345 X [1.01]

  FP = 348.45

Reconciling FP and LOC

LANGUAGE	*AVERAGE SOURCE STATEMENTS PER FUNCTION POINT*
1032/AF	16
1st Generation default	320
2nd Generation default	107
3rd Generation default	80
4th Generation default	20
5th Generation default	5
Assembly (Basic)	320
BASIC	107
C	128
C++	53
COBOL	107
JAVA	53
Visual Basic 5	29

What would be LOC for the pervious example in JAVA: **348.45**

348 x 53 = 18457 LOC